



Operating system and system programming (25CS2104E)

Process Termination Messages

Review-I

Under the Esteemed Guidance of

Raghupathi M
CSE Department

Team Members:

Name-	Roll Number:
A.Shashank Reddy	2520090139
D.Vignesh Kumar	2520090071
Y.Sharath Goud	2520090062



Contents

- Abstract
- Introduction
- Hardware and Software requirements
- System Architecture
- Conclusion



Abstract

Every process, once created, eventually comes to an end - either by completing its task normally or by being forced to stop abnormally. When a process terminates, the operating system does not simply discard it; it generates a **termination message** or exit status that records how and why the process ended.

This project studies how the operating system creates, stores, and delivers process termination messages, covering normal termination through *exit()*, abnormal termination via signals. A small C program on a Linux system is used to demonstrate these concepts practically.

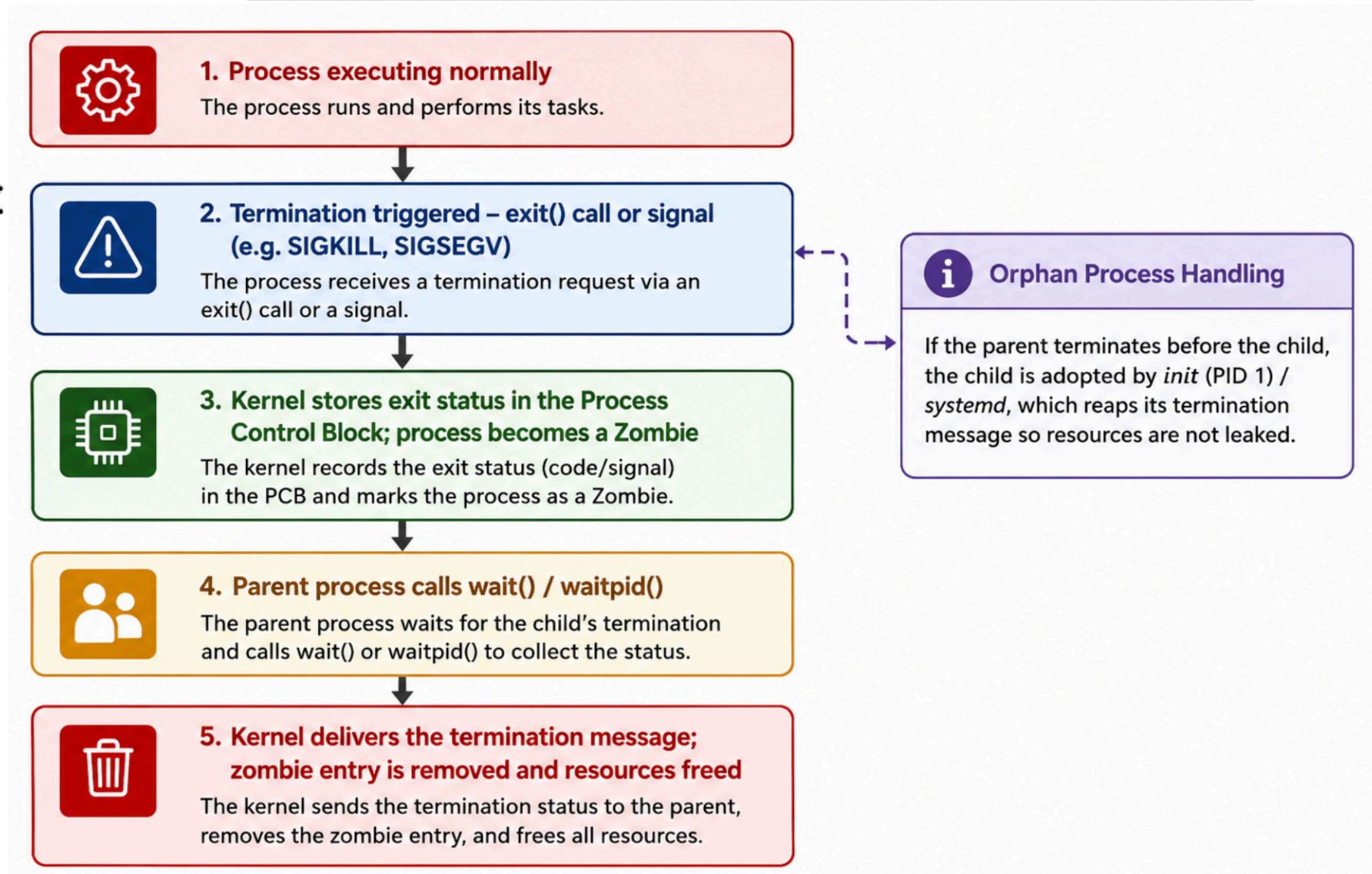
Introduction

- **Process termination** is the final stage in a process's lifecycle, where the OS reclaims its resources (memory, PCB, open files) after execution ends.
- **Normal termination** occurs when a process completes its task and calls *exit()*, returning a status code to the parent.
- **Abnormal termination** happens when a process is killed by a signal (e.g., *SIGKILL*, *SIGSEGV*) due to an error or external request.
- **Termination messages** (exit codes / signal numbers) tell the parent process whether the child succeeded, failed, or was interrupted, enabling error handling and process coordination.
- **Scope:** this project implements and demonstrates termination-message generation and retrieval using *fork()*, *exit()*, *wait()* and *waitpid()* on Linux.

Hardware and Software requirements

Category	Requirement	Specification
Hardware	Processor	Intel i3 or higher (or equivalent AMD)
Hardware	RAM	4 GB minimum (8 GB recommended)
Hardware	Storage	20 GB free disk space
Software	Operating System	Linux (Ubuntu 20.04+) / Windows 10 with WSL
Software	Compiler	GCC (GNU Compiler Collection)
Software	Editor / IDE	VS Code / vi / gedit
Software	Language	C Programming Language

System Architecture





Conclusion

Process termination messages are an essential communication link between a child process and its parent, conveying the exit status or the signal that caused termination.

- The kernel reliably records this information in the Process Control Block, and system calls such as `wait()` and `waitpid()` let the parent retrieve it cleanly.
- Proper handling of zombie and orphan processes prevents resource leaks and keeps the process table clean, which is vital for system stability.
- The demonstration program confirmed that both normal and signal-based termination messages can be captured and interpreted correctly, reinforcing core operating-system concepts of process management.



Thank you